

Build Task — Graph Skeleton (feeds into Project 3) — Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper LangGraph — how a real codebase would actually structure this, so Document 10 can build on it). Read Basic first even if you already know LangGraph — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — What you're building, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What you're building, and the exact pieces

Basic Version

You're translating your Project 2 agent (the Doc07 tool-calling loop) into a `StateGraph` with 5 requirements: a typed state, at least one real branch (not just a straight line), a checkpointer so state survives a pause, one `interrupt()` point standing in for a human check, and the same tool-calling behavior your Doc07 loop already has.

Split this across 4 small files, each doing one job:

- `state.py` — just the `TypedDict` describing what the graph tracks.
- `nodes.py` — the functions that do the actual work (think, act, a risky-action placeholder).
- `graph.py` — wires nodes and edges together, connects the checkpointer, returns a compiled graph.
- `test_graph.py` — proves it actually works, including the pause/resume cycle.

Here are the exact pieces you need to look up and use:

- `TypedDict`, `Annotated[list[str], operator.add]` — your state shape, with a reducer for anything that should accumulate (like your scratchpad).
- `StateGraph`, `START`, `END`, `add_node`, `add_edge`, `add_conditional_edges` — the graph itself.
- `MemorySaver` from `langgraph.checkpoint.memory` — the checkpointer.
- `interrupt` and `Command` from `langgraph.types` — the pause/resume mechanism.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Think of this Build Task as 3 things you've already practiced separately, combined into one graph: the Real-world exercise's loop-to-graph translation (think/act nodes, a conditional edge deciding "call a tool again" vs. "finish"), and the Failure exercise's checkpointer + `interrupt()` (a saved, pausable run). This task is genuinely just those 2 things, wired into the same graph, plus one more node standing in for a "risky action" that needs human approval before it runs.

The exact pieces, file by file:

- **state.py**: `class AgentState(TypedDict): task: str; scratchpad: Annotated[list[str], operator.add]; next_action: str; final_answer: str; approved: bool` — everything any node might need to read or write, in one place.
- **nodes.py**: `think(state)` (calls the model, decides tool-vs-finish, same as your Doc07 loop), `act(state)` (runs the chosen tool, reusing your Doc07 tool functions), `risky_action(state)` (calls `interrupt(...)` to pause for approval, then does the placeholder "risky" thing once resumed), `route(state)` (reads `next_action`, returns which path to take).
- **graph.py**: `def build_graph() -> CompiledStateGraph:` — builds the `StateGraph`, adds all nodes, wires `add_conditional_edges("think", route, {...})`, connects a `MemorySaver()` checkpointer via `builder.compile(checkpointer=...)`, and returns the compiled graph so other files (and Document 10) can just call `build_graph()` and get a ready-to-use graph, without repeating any wiring logic.
- **test_graph.py**: at least one test proving the no-risky-action path finishes cleanly, one proving the interrupt path actually pauses, and one proving a resume with the same `thread_id` continues correctly.

Constraint to keep in mind while writing every node: "no node may quietly swallow an error."

Wrap risky operations in `try / except`, but always either re-raise, or record the failure visibly in state (e.g., an `error` field) — never a bare `except: pass`.

Think about who calls `build_graph()`, and how many times. Document 10 is going to import this exact skeleton and build the full Project 3 app on top of it — which means `graph.py` needs to be genuinely reusable, not a script that only works when run top-to-bottom once. Two questions worth asking now, while it's cheap to fix:

- **Does `build_graph()` create a new checkpointer every time it's called, or share one?** If something calls `build_graph()` twice expecting to resume the same paused run, a fresh `MemorySaver()` each time would silently break that — the second call's checkpointer has no memory of the first call's paused state. Decide on purpose whether `build_graph()` takes a

checkpointer as a parameter (more flexible — the caller decides), or owns a module-level one (simpler, but only correct if there's ever exactly one shared graph instance in the whole program).

- **Does the risky-action node distinguish "not yet approved" from "explicitly rejected"?** A real human-in-the-loop gate needs a real rejection path, not just an approval path — `interrupt()` resuming with `Command(resume=False)` should lead somewhere sensible (skip the risky action, record why), not crash or silently proceed as if it had been approved.

The extra piece that answers the first question — a parameterized `build_graph()` :

```
def build_graph(checkpointer=None):
    if checkpointer is None:
        checkpointer = MemorySaver()
    builder = StateGraph(AgentState)
    # ... add_node / add_edge / add_conditional_edges ...
    return builder.compile(checkpointer=checkpointer)
```

This lets `test_graph.py` (and later, Document 10's app) pass in its own checkpointer — a `MemorySaver()` for quick tests, a real persisted one for production — without touching `graph.py` itself.

Difference between Basic and Intermediate: Basic names the 4 files, their jobs, and the exact tools. Intermediate connects this task explicitly to the 2 exercises it's built from, spells out each file's real content, and asks the harder design question underneath the whole task — not "does the graph work once," but "is this skeleton actually safe for Document 10 to import and build on," which is the literal stated purpose of this Build Task — and answers it with a parameterized checkpointer and a real rejection path, not just an approval path.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
state.py:
    AgentState: task, scratchpad (accumulates), next_action, final_answer, approved

nodes.py:
    think(state): call the model; decide "call_tool" or "finish";
                  append to scratchpad
    act(state): run the chosen tool; append the result to scratchpad
    risky_action(state): call interrupt() to pause for a human;
                       once resumed, do the placeholder action
    route(state): return state["next_action"]

graph.py:
    function build_graph():
        make a StateGraph(AgentState)
        add think, act, risky_action as nodes
        start -> think
        think, conditional on route:
            "call_tool" -> act, "risky" -> risky_action, "finish" -> end
        act -> think (loop back)
        risky_action -> end
        compile with a MemorySaver checkpointer
        return the compiled graph

test_graph.py:
    test 1: a task with no risky action finishes without ever pausing
    test 2: a task that reaches risky_action pauses (check
            __interrupt__ in the result)
    test 3: resuming with the same thread_id continues and
            finishes correctly
```

The trickiest part of turning that plan into real code — the risky-action node itself:

```
from langgraph.types import interrupt

def risky_action(state):
    question = f"Approve this risky action for task: {state['task']}?"
```

```
approval = interrupt({"question": question})
if not approval:
    return {"final_answer": "Risky action was not approved.", "approved": False}
# placeholder for the real risky action (e.g. send an email, spend money)
return {"final_answer": "Risky action completed.", "approved": True}
```

Expected output if you run just this (nothing calls it yet): nothing — this is one node function; it only does anything once it's wired into a compiled graph and invoked.

Try finishing the rest yourself before looking at the Solution.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
state.py:
    from typing import TypedDict, Annotated
    import operator

    class AgentState(TypedDict):
        task: str
        scratchpad: Annotated[list[str], operator.add]
        next_action: str
        final_answer: str
        approved: bool

nodes.py:
    function think(state: AgentState) -> dict:
        call your Doc07 model function with state["task"] and state["scratchpad"]
        if it wants a tool: return scratchpad update, next_action="call_tool"
        if it wants the risky action: return scratchpad update, next_action="risk
        otherwise: return scratchpad update, next_action="finish", final_answer

    function act(state: AgentState) -> dict:
        run the chosen tool (reusing your Doc07 tool functions)
        return the observation as a scratchpad update
        never swallow a tool error silently -- catch it, record it
        in scratchpad, re-raise or route to a clear failure state

    function risky_action(state: AgentState) -> dict:
        approval = interrupt({"question": "Approve this risky action?"})
        if not approval:
            return final_answer explaining it was rejected, approved=False
        otherwise:
            do the placeholder risky thing, return final_answer, approved=True

    function route(state: AgentState) -> str:
        return state["next_action"]

graph.py:
    function build_graph() -> CompiledStateGraph:
        builder = StateGraph(AgentState)
        builder.add_node("think", think)
        builder.add_node("act", act)
        builder.add_node("risky_action", risky_action)
```

```

        builder.add_edge(START, "think")
        builder.add_conditional_edges("think", route, {
            "call_tool": "act", "risky": "risky_action", "finish": END,
        })
        builder.add_edge("act", "think")
        builder.add_edge("risky_action", END)
        checkpointer = MemorySaver()
        return builder.compile(checkpointer=checkpointer)

test_graph.py:
graph = build_graph()
config = {"configurable": {"thread_id": "test-1"}}

# a task that never needs the risky action
starting_state = {
    "task": "...", "scratchpad": [], "next_action": "",
    "final_answer": "", "approved": False,
}
result = graph.invoke(starting_state, config=config)
assert "__interrupt__" not in result

# a task that does need it
config2 = {"configurable": {"thread_id": "test-2"}}
paused = graph.invoke({"task": "risky task", ...}, config=config2)
assert "__interrupt__" in paused

resumed = graph.invoke(Command(resume=True), config=config2)
assert resumed["approved"] is True

```

Turning that plan into real code, here's the piece worth seeing on its own first — `route`'s mapping needs a real path for every value `think` can actually produce, `END` included directly in the mapping (a conditional edge can point straight at `END`, not just at another node):

```

builder.add_conditional_edges(
    "think",
    route,
    {"call_tool": "act", "risky": "risky_action", "finish": END},
)

```

Try finishing the rest yourself before looking at the full Solution below.

Once that's working, make `build_graph()` take its checkpointer as a parameter, and give `risky_action` a real rejection path:

```

graph.py:
    function build_graph(checkpointer=None) -> CompiledStateGraph:
        if checkpointer is None: checkpointer = MemorySaver()
        ... same node/edge wiring as above ...
        return builder.compile(checkpointer=checkpointer)

nodes.py -- risky_action, now with a real rejection path:
    function risky_action(state):
        approval = interrupt({"question": ...})
        if approval is not True:
            # explicit rejection, not a crash and not silent approval
            return {
                "final_answer": "Risky action was not approved.",
                "approved": False,
            }
        try:
            # placeholder for the real action
            outcome = "Risky action completed."
        except Exception as e:
            # never swallow this -- record it visibly
            return {"final_answer": f"Risky action failed: {e}", "approved": False}
        return {"final_answer": outcome, "approved": True}

test_graph.py:
    also test the rejection path explicitly:
        paused = graph.invoke({...}, config=config3)
        resumed = graph.invoke(Command(resume=False), config=config3)
        assert resumed["approved"] is False
        assert "not approved" in resumed["final_answer"]

```

Notice the shape: `build_graph(checkpointer=None)` lets callers (your own tests, or Document 10's app later) supply their own checkpointer, instead of `graph.py` silently owning one `MemorySaver()` that every caller is forced to share. And `risky_action` now has 3 real outcomes — approved, explicitly rejected, and failed — instead of just "approved or crash."

The parameterized `build_graph`, made real — this is the one part most worth getting right before Document 10 needs it:

```

from langgraph.checkpoint.memory import MemorySaver

def build_graph(checkpointer=None):
    if checkpointer is None:
        checkpointer = MemorySaver()
    builder = StateGraph(AgentState)
    builder.add_node("think", think)

```

```

builder.add_node("act", act)
builder.add_node("risky_action", risky_action)
builder.add_edge(START, "think")
builder.add_conditional_edges(
    "think", route, {"call_tool": "act", "risky": "risky_action", "finish": END
)
builder.add_edge("act", "think")
builder.add_edge("risky_action", END)
return builder.compile(checkpointer=checkpointer)

```

Expected behavior: `build_graph()` with no arguments still works exactly as in Intermediate — the default `MemorySaver()` is only created when nothing is passed in.

`build_graph(checkpointer=my_sqlite_saver)` uses that instead, with zero other changes to the function.

Fill in the rejection path and the rest of `test_graph.py` yourself, then compare both of your finished versions against the [Solution](#).

Difference between Basic and Intermediate: Basic's pseudocode gets one working risky-action node with an approval path. Intermediate wires the complete graph across all 4 files, with a real conditional edge mapping (including a direct route to `END`) and a real test file covering all required behaviors — and additionally makes `build_graph()` accept its checkpointer instead of always creating its own, and gives `risky_action` a genuine 3-way outcome (approved / rejected / failed) instead of only handling the happy path — both of which matter the moment this skeleton gets imported and reused by Document 10, exactly as the Build Task's Goal says it will be.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Solution

Every code block below shows the exact output you'd see if you ran it, right after the code.

`call_model_with_tools` and `run_tool` stand in for your real Doc07 functions, written as small deterministic stubs so the whole example is runnable without a live API key — in your own Build Task, import and call your actual Project 2 code instead. Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

```
# stand-ins for your real Doc07 code -- replace with your actual imports
def call_model_with_tools(task: str, scratchpad: list[str]) -> dict:
    if not scratchpad:
        if "risky" in task:
            return {"action": "risky"}
        if "*" in task:
            return {"action": "call_tool", "tool_name": "calculator"}
    return {"action": "finish", "answer": "84"}

def run_tool(tool_name: str, task: str) -> str:
    return "84" if tool_name == "calculator" else "unknown tool"
```

Basic Version

Approach 1 — the direct way, 1 shared checkpointer

Story — `state.py` / `nodes.py` / `graph.py` / `test_graph.py` : this is Project 3's graph skeleton — the Real-world exercise's loop-to-graph translation and the Failure exercise's checkpointer/interrupt cycle, combined into one graph. **If not:** Document 10 would be the first place any of these pieces ever had to work together, with no smaller version to trust.

`state.py`

```
from typing import TypedDict, Annotated
import operator

class AgentState(TypedDict):
    task: str
    scratchpad: Annotated[list[str], operator.add]
    next_action: str
```

```
final_answer: str
approved: bool
```

nodes.py

```
from langgraph.types import interrupt
from state import AgentState

def think(state: AgentState) -> dict:
    decision = call_model_with_tools(state["task"], state["scratchpad"])
    action = decision["action"]
    if action == "call_tool":
        note = f"Thought: calling {decision['tool_name']}"
        return {"scratchpad": [note], "next_action": "call_tool"}
    if action == "risky":
        note = "Thought: this needs approval"
        return {"scratchpad": [note], "next_action": "risky"}
    return {
        "scratchpad": ["Thought: done"],
        "next_action": "finish",
        "final_answer": decision["answer"],
    }

def act(state: AgentState) -> dict:
    result = run_tool("calculator", state["task"])
    return {"scratchpad": [f"Observation: {result}"]}

def risky_action(state: AgentState) -> dict:
    question = f"Approve this risky action for task: {state['task']}?"
    approval = interrupt({"question": question})
    if not approval:
        return {"final_answer": "Risky action was not approved.", "approved": False}
    return {"final_answer": "Risky action completed.", "approved": True}

def route(state: AgentState) -> str:
    return state["next_action"]
```

graph.py

```
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from state import AgentState
```

```

from nodes import think, act, risky_action, route

def build_graph():
    builder = StateGraph(AgentState)
    builder.add_node("think", think)
    builder.add_node("act", act)
    builder.add_node("risky_action", risky_action)
    builder.add_edge(START, "think")
    builder.add_conditional_edges(
        "think", route, {"call_tool": "act", "risky": "risky_action", "finish": END}
    )
    builder.add_edge("act", "think")
    builder.add_edge("risky_action", END)
    return builder.compile(checkpointer=MemorySaver())

```

test_graph.py

```

from langgraph.types import Command
from graph import build_graph

graph = build_graph()

# no risky action needed
config1 = {"configurable": {"thread_id": "test-1"}}
task1_start = {
    "task": "What is 12 * 7?", "scratchpad": [], "next_action": "",
    "final_answer": "", "approved": False,
}
result1 = graph.invoke(task1_start, config=config1)
print("no-risk result:", result1["final_answer"])

# risky action -- pauses, then resumes
config2 = {"configurable": {"thread_id": "test-2"}}
task2_start = {
    "task": "do a risky thing", "scratchpad": [], "next_action": "",
    "final_answer": "", "approved": False,
}
paused = graph.invoke(task2_start, config=config2)
print("paused, has interrupt:", "__interrupt__" in paused)

resumed = graph.invoke(Command(resume=True), config=config2)
print("resumed:", resumed["final_answer"], resumed["approved"])

```

Expected output:

```
no-risk result: 84  
paused, has interrupt: True  
resumed: Risky action completed. True
```

This meets every Build Task requirement: a typed state, a real conditional edge (`think` branches 3 ways), a checkpoint, one `interrupt()` point, and the same tool-calling shape as a Doc07 loop. It's missing the parameterized checkpointer and the explicit rejection test — both fine for a first working version.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — errors surfaced in state, not swallowed

Story — `nodes.py` / `test_graph.py` (Intermediate): Basic proved the graph works; this version makes it safe to build on — a tool failure gets recorded in state instead of crashing the node, per the Build Task's own constraint. **If not:** the first real tool failure in Project 3 would crash the whole graph instead of routing to a clear, logged failure state.

`nodes.py` (only the parts that change from Basic)

```
from langgraph.types import interrupt
from state import AgentState

def think(state: AgentState) -> dict:
    decision = call_model_with_tools(state["task"], state["scratchpad"])
    action = decision["action"]
    if action == "call_tool":
        note = f"Thought: calling {decision['tool_name']}"
        return {"scratchpad": [note], "next_action": "call_tool"}
    if action == "risky":
        note = "Thought: this needs approval"
        return {"scratchpad": [note], "next_action": "risky"}
    return {
        "scratchpad": ["Thought: done"],
        "next_action": "finish",
        "final_answer": decision["answer"],
    }

def act(state: AgentState) -> dict:
    try:
        result = run_tool("calculator", state["task"])
    except Exception as e:
        # why: never swallow it -- put the failure directly in state,
        # where think() and the final result can actually see it
        return {
            "scratchpad": [f"Observation: TOOL FAILED: {e}"],
            "next_action": "finish",
            "final_answer": f"Tool failed: {e}",
        }
```

```

    return {"scratchpad": [f"Observation: {result}"]}

def risky_action(state: AgentState) -> dict:
    question = f"Approve this risky action for task: {state['task']}?"
    approval = interrupt({"question": question})
    if not approval:
        return {"final_answer": "Risky action was not approved.", "approved": False}
    return {"final_answer": "Risky action completed.", "approved": True}

def route(state: AgentState) -> str:
    return state["next_action"]

```

test_graph.py, with a real rejection test added:

```

from langgraph.types import Command
from graph import build_graph

graph = build_graph()

config1 = {"configurable": {"thread_id": "test-1"}}
task1_start = {
    "task": "What is 12 * 7?", "scratchpad": [], "next_action": "",
    "final_answer": "", "approved": False,
}
result1 = graph.invoke(task1_start, config=config1)
# why: a clean task must never pause -- catches a routing bug early
assert "__interrupt__" not in result1
assert result1["final_answer"] == "84"

config2 = {"configurable": {"thread_id": "test-2"}}
task2_start = {
    "task": "do a risky thing", "scratchpad": [], "next_action": "",
    "final_answer": "", "approved": False,
}
paused = graph.invoke(task2_start, config=config2)
assert "__interrupt__" in paused # how: must pause before the risky action runs

approved = graph.invoke(Command(resume=True), config=config2)
assert approved["approved"] is True

config3 = {"configurable": {"thread_id": "test-3"}}
task3_start = {
    "task": "do a risky thing", "scratchpad": [], "next_action": "",
    "final_answer": "", "approved": False,
}

```

```

}
graph.invoke(task3_start, config=config3)
rejected = graph.invoke(Command(resume=False), config=config3)
assert rejected["approved"] is False
assert "not approved" in rejected["final_answer"]

print("All test_graph.py assertions passed.")

```

Expected output:

```
All test_graph.py assertions passed.
```

Difference from Basic: `act` no longer lets a tool exception crash the node silently past detection — it's caught, recorded directly in `scratchpad` and `final_answer`, and routed straight to `"finish"` instead of continuing as if nothing happened, satisfying the Build Task's "no node may quietly swallow an error" constraint. `test_graph.py` also now tests the **rejection** path explicitly (`Command(resume=False)`), not just the approval path — both are real, expected outcomes of a human-in-the-loop gate.

Approach 2 — a parameterized `build_graph`, ready for Document 10

Story: Document 10 is going to import this exact skeleton and build the full Project 3 app on top of it — `graph.py` needs to be genuinely reusable, not a script that only works run top-to-bottom once. **If not:** a hard-coded `MemorySaver()` inside `graph.py` would silently break the moment a second caller (a test file, Document 10's app) needed its own independent checkpointer.

`graph.py`

```

from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from state import AgentState
from nodes import think, act, risky_action, route

def build_graph(checkpointer=None):
    """Build the Project 3 graph skeleton.

    Pass a checkpointer explicitly (e.g. a persisted one) for anything
    beyond quick local testing -- a fresh MemorySaver() is only created
    if none is given.
    """
    if checkpointer is None:
        checkpointer = MemorySaver() # why: never force every caller to share one

    builder = StateGraph(AgentState)

```

```

builder.add_node("think", think)
builder.add_node("act", act)
builder.add_node("risky_action", risky_action)
builder.add_edge(START, "think")
builder.add_conditional_edges(
    "think", route, {"call_tool": "act", "risky": "risky_action", "finish": END
)
builder.add_edge("act", "think")
builder.add_edge("risky_action", END)
return builder.compile(checkpointer=checkpointer)

```

test_graph.py, now supplying its own checkpointer explicitly (so tests never depend on `graph.py`'s default):

```

from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command
from graph import build_graph

checkpointer = MemorySaver()
graph = build_graph(checkpointer=checkpointer)

config1 = {"configurable": {"thread_id": "test-1"}}
task1_start = {
    "task": "What is 12 * 7?", "scratchpad": [], "next_action": "",
    "final_answer": "", "approved": False,
}
result1 = graph.invoke(task1_start, config=config1)
assert "__interrupt__" not in result1
assert result1["final_answer"] == "84"

config2 = {"configurable": {"thread_id": "test-2"}}
task2_start = {
    "task": "do a risky thing", "scratchpad": [], "next_action": "",
    "final_answer": "", "approved": False,
}
graph.invoke(task2_start, config=config2)
approved = graph.invoke(Command(resume=True), config=config2)
assert approved["approved"] is True

config3 = {"configurable": {"thread_id": "test-3"}}
task3_start = {
    "task": "do a risky thing", "scratchpad": [], "next_action": "",
    "final_answer": "", "approved": False,
}
graph.invoke(task3_start, config=config3)
rejected = graph.invoke(Command(resume=False), config=config3)

```

```
assert rejected["approved"] is False

print("All assertions passed, using an explicitly supplied checkpointer.")
```

Expected output:

```
All assertions passed, using an explicitly supplied checkpointer.
```

Approach 3 — swapping in a real persisted checkpointer, with zero other changes

Story: Approach 2's whole point is that swapping in a real, persisted checkpointer should cost one line at the call site — this proves it. **If not:** Document 10 would need to edit `graph.py` itself the day it needs a checkpointer that survives a restart, instead of just changing what it passes in.

```
from langgraph.checkpoint.sqlite import SqliteSaver
from graph import build_graph

with SqliteSaver.from_conn_string("project3_checkpoints.sqlite") as checkpointer:
    graph = build_graph(checkpointer=checkpointer)
    config = {"configurable": {"thread_id": "real-run-1"}}
    task_start = {
        "task": "do a risky thing", "scratchpad": [], "next_action": "",
        "final_answer": "", "approved": False,
    }
    graph.invoke(task_start, config=config)
    print("Paused and saved to disk -- this state now survives past this process.")
```

Expected output:

```
Paused and saved to disk -- this state now survives past this process.
```

Nothing in `graph.py`, `nodes.py`, or `state.py` changed — `build_graph(checkpointer=...)` from Approach 2 is exactly what makes this swap possible with a one-line change at the call site.

Difference from Approach 1, and between Approaches 2/3: Approach 1's `build_graph()` always owns its own `MemorySaver()`, so every caller shares whatever instance was created inside `graph.py` — fine for one script, fragile the moment two different callers (a test file and Document 10's app) need genuinely independent graphs, or one of them needs a real persisted checkpointer. Approach 2 fixes that by accepting a checkpointer as a parameter. Approach 3 doesn't change any code at all — it's proof that Approach 2's one change is exactly the seam Document 10 needs: swap `MemorySaver()` for `SqliteSaver` (or a real database-backed saver in production) at the call site, with the graph skeleton itself untouched.

Which one should you actually write? Approach 1 already satisfies every literal Build Task requirement — ship it first if you're short on time. Move to Approach 2's error-surfacing in `act` and the explicit rejection test as soon as you're done with the happy path — the "no node may quietly swallow an error" constraint is explicit in the Goal, and a human-in-the-loop gate genuinely needs a tested rejection path, not just an approval one. Reach for Approach 3's parameterized `build_graph()` before you consider this skeleton actually done — the Build Task's whole stated purpose is that Document 10 imports and builds on it, and a hard-coded `MemorySaver()` inside `graph.py` is exactly the kind of thing that quietly breaks the moment a second caller needs its own checkpoint.